DATABRICKS • DAY 19

Lakeflow Jobs

Databricks Workflows / Jobs Orchestration

OFFICIALLY VERIFIED

Current Databricks documentation calls the orchestration product Lakeflow Jobs. It coordinates and runs one or more tasks as part of a larger workflow, including ETL, notebooks, machine-learning workloads, SQL, dbt, and other supported task types.

1. The problem Lakeflow Jobs solves

Imagine you have separate pieces of a production pipeline: ingest raw data, clean it, validate it, build a Gold table, and refresh or notify downstream consumers. Writing each piece is only half the problem. Production also needs ordering, scheduling, dependencies, retries, monitoring, parameters, and failure handling.

Lakeflow Jobs is the orchestration layer that coordinates those pieces as a repeatable workflow.

Trigger → **Job** → **Tasks** → **Compute** → **Monitor**

2. The three core ideas

Concept	Meaning	Simple mental model
Job	The complete workflow definition containing one or more tasks.	The full movie production.
Task	An individual unit of work inside the job.	One crew activity: lights, camera, or shooting.
Trigger	What starts a run: manual, scheduled, or supported automated trigger.	The call time that starts production.

INTERVIEW ONE-LINER

A Job is the workflow; Tasks are the units of work inside it; dependencies define execution order; triggers start runs; and monitoring tells you what happened.

3. Job vs Task: understand this clearly

A common beginner mistake is to use the words job and task interchangeably. In Lakeflow Jobs, the job is the container/orchestration definition. Each task represents something that actually executes.



Tasks can depend on upstream tasks. Databricks represents the job graph as a directed acyclic graph (DAG), which makes the execution order and branching visible.

4. Dependencies and the DAG

- A task can depend on one or more upstream tasks.
- Independent tasks can run in parallel when their dependency conditions allow it.
- The graph must remain acyclic: dependencies cannot create a circular loop such as A → B → A.
- The DAG is useful both for authoring the workflow and for understanding execution status during a run.

EXAMPLE

Bronze ingestion finishes first. Two independent Silver transformations can then run in parallel. A validation task waits for both. Gold runs only when the required upstream tasks satisfy their run conditions.

5. What can a task run?

Task family	Typical use	Example
Notebook	Execute notebook logic	Transform Silver data
Python	Run Python script or packaged Python code	Reusable ingestion utility
SQL	Run supported SQL assets through Databricks SQL	Execute validation query
Pipeline	Run a Lakeflow Spark Declarative Pipeline	Bronze/Silver pipeline
dbt	Run dbt Core transformations	Build modeled tables
Run Job	Invoke another Lakeflow Job	Compose larger workflows
Control flow	If/else and For each	Branch or iterate over tables

IMPORTANT

The exact compute required depends on the task type. For example, SQL tasks use a supported SQL warehouse, while other tasks can use serverless jobs compute or configured Databricks compute depending on the task and workspace capabilities.

6. Triggers: how a job starts

A job definition does nothing until a run is started. Lakeflow Jobs supports running on demand and configuring automated execution patterns. Scheduling is one of the most common production patterns.

Trigger pattern	When useful	Example
Manual / Run now	Development, testing, ad-hoc reruns	Engineer starts the job after a code change
Scheduled	Predictable batch workloads	Every day at 2:00 AM
Automated/event-oriented triggers	When execution should react to supported conditions	Start when a supported upstream condition occurs

7. Control flow: beyond A → B → C

Modern Lakeflow Jobs supports control-flow tasks, which means a workflow does not have to be a single straight line.

- If/else: evaluate an expression and direct downstream execution based on the result.
- For each: execute a nested task repeatedly using a different input for each iteration. A classic use case is applying the same transformation to a list of tables.

> **QUALITY-GATE EXAMPLE**
> Quality Check → PASS → publish Gold table. Quality Check → FAIL → validation/remediation path. This is a useful way to explain orchestration in interviews because it shows that production pipelines need decision logic, not only scheduling.

8. Parameters and passing values

Jobs become reusable when values are parameterized instead of hard-coded. Parameters can be used to pass runtime context such as environment, date, table name, or batch identifier to tasks. Task values can also help pass information between tasks in a workflow.

- Example: process_date = 2026-09-08
- Example: environment = dev / test / prod
- Example: table_name supplied to a For each iteration

> **DESIGN PRINCIPLE**
> Orchestration should describe the workflow and its execution behavior. Business transformation logic should normally remain in notebooks, Python packages, SQL, dbt, or pipelines rather than becoming a giant orchestration definition.

9. Compute: orchestration and execution are different things

Lakeflow Jobs orchestrates work, but each executable task still needs an execution environment. This distinction is important: the Job decides what should run and in what order; compute is where the code actually runs.

Job defined → **Task selected** → **Compute assigned** → **Task executes** → **Result recorded**

Depending on the task type and workspace configuration, tasks may use serverless jobs compute, configured job compute, SQL warehouses, or other supported compute. Databricks increasingly recommends serverless compute for eligible jobs because it reduces infrastructure management.

10. Monitoring and run history

- View job run status and individual task status.
- Inspect duration, outputs, logs, and error details for failed tasks.
- Use notifications and observability features to surface failures and operational events.
- For account-level analysis, Databricks exposes job and task information through Lakeflow system tables in supported environments.

11. Repair runs

If a multi-task job fails late in the workflow, rerunning everything from the beginning may be wasteful. Databricks supports repair runs for failed or cancelled jobs. A repair lets you rerun failed and dependent work without necessarily repeating successful upstream tasks.

> **EXAMPLE**
> Ingest [OK] -> Transform [OK] -> Report [FAILED]. After fixing the Report issue, use a repair run so the workflow can continue from the affected portion instead of needlessly repeating the successful ingestion and transformation steps.

12. Lakeflow Jobs vs an external orchestrator

Scenario	Likely fit	Why
Mostly Databricks workloads	Lakeflow Jobs	Native authoring, scheduling, task graph, monitoring, and repair in the same platform
Cross-platform enterprise workflow	External orchestrator may be useful	One workflow may need to coordinate Databricks plus many external systems
External orchestrator triggering Databricks	Hybrid	Tools such as Airflow can trigger Databricks jobs while Databricks executes its workloads

13. End-to-end production example

Suppose an e-commerce team builds a daily sales pipeline. A sensible Lakeflow Job could look like this:



- Task 1: ingest orders from the source.
- Task 2: ingest customer data. It can run independently of Task 1 if no dependency exists.
- Task 3: transform and join the required datasets after both ingestion tasks finish.
- Task 4: run data-quality checks.
- Task 5: publish Gold tables only on the success path.
- Task 6: notify or trigger downstream consumption.

The job can be scheduled daily, parameterized with the processing date, monitored at task level, and repaired from a failed point if something goes wrong.

14. Common mistakes

Mistake	Better approach
Treating Job and Task as the same thing	Job = orchestration container; Task = unit of work.
Making every task depend on the previous one	Allow independent tasks to run concurrently where safe.
Hard-coding dates and environments	Use parameters to make workflows reusable.
Putting all business logic into orchestration	Keep transformation logic in code/pipelines and use Jobs for orchestration.
Rerunning an expensive pipeline from scratch after a late failure	Use repair runs when appropriate.

15. Practical checklist for creating a Job

- Define the outcome of the workflow.
- Break the workflow into clear tasks.
- Choose a task type for each unit of work.
- Define task dependencies and run conditions.
- Choose/allow the appropriate compute for each task.
- Add parameters instead of hard-coded runtime values.
- Configure trigger or schedule.
- Configure retries, timeouts, notifications, and other operational settings as needed.
- Test failure handling and understand how to repair a run.
- Monitor the workflow after deployment.

16. Interview Q&A

Q1. What is Lakeflow Jobs?

Lakeflow Jobs is Databricks workflow automation/orchestration for coordinating and running multiple tasks as part of larger data and AI workflows.

Q2. What is the difference between a Job and a Task?

A Job represents the workflow definition. A Task is an individual unit of work inside that Job.

Q3. What is a DAG in Lakeflow Jobs?

The DAG is the directed acyclic graph of tasks and their dependencies. It shows the order and branching of the workflow without circular dependencies.

Q4. Can Lakeflow Jobs run only notebooks?

No. It supports multiple task types, including notebooks, Python, SQL, pipelines, dbt, Run Job, and control-flow tasks such as If/else and For each.

Q5. What happens if a task fails?

The run and task state can be inspected through monitoring. Depending on configuration, retries may occur, and failed or cancelled jobs can be repaired and rerun from the affected workflow portion.

Q6. Why use Lakeflow Jobs instead of manually running notebooks?

Production requires repeatability, ordering, scheduling, dependencies, parameterization, observability, and failure handling. Lakeflow Jobs provides that orchestration layer.

Q7. Lakeflow Jobs vs Airflow?

Use Lakeflow Jobs naturally when the workflow is mainly inside Databricks. An external orchestrator can be useful for broad cross-platform coordination and can also trigger Databricks jobs.

17. Five words to remember

JOB • TASK • DEPENDENCY • TRIGGER • MONITORING

If you can explain these five concepts and connect them to compute, parameters, failures, and repair runs, you understand the core of Databricks workflow orchestration.

18. Official references used for verification

- **Lakeflow Jobs overview:** https://docs.databricks.com/aws/en/jobs
- **Configure and edit Lakeflow Jobs:** https://docs.databricks.com/aws/en/jobs/configure-job
- **Configure and edit tasks:** https://docs.databricks.com/aws/en/jobs/configure-task
- **For each task:** https://docs.databricks.com/aws/en/jobs/tasks/for-each
- **SQL task:** https://docs.databricks.com/aws/en/jobs/tasks/sql
- **dbt transformations in Lakeflow Jobs:** https://docs.databricks.com/aws/en/jobs/how-to/use-dbt-in-workflows

NEXT: Day 20 — Job Compute vs All-Purpose Compute